Liquidium Cross-Chain Lending

Security Assessment

November 14, 2025

Prepared for:
Peter Giammanco
Liquidium

Prepared by: **Bo Henderson and Tjaden Hess**

Table of Contents

Table of Contents 1
Project Summary 3
Executive Summary 4
Project Goals 6
Project Targets 7
Project Coverage 8
Codebase Maturity Evaluation 10
Summary of Findings 12
Detailed Findings 14
1. Lack of authorization check in withdraw and add_account methods 14
2. Interest calculations allow pool funds to be drained 16
3. Withdrawal rollback may fail if all funds are withdrawn 18
4. User funds could be trapped due to timeout race 21
5. Liquidations may be refunded twice 23
6. Failed borrow produces WithdrawalFailed notification 25
7. Stale prices cannot be updated 27
8. Lack of a delimiter in asset pair symbols 28
9. Liquidator funds become stuck when min_collateral_amount cannot be satisfied 29
10. Borrow checks do not account for pending borrows 31
11. Protocol reports incorrect outflow status on borrow failure 34
12. BTC deposits can be front-run 36
13. Improper BTC withdrawal status checking may allow double spends and lost funds 37
14. Pool event notifications may be sent twice 40
15. Repayment interest waiver calculations violate invariants 42
16. High transaction volume can cause deposits to be dropped 44
17. Liquidation search time grows quadratically with total number of users 46
18. Removal of an account iterates over all accounts 49
19. ERC-20 deposits are not compatible with smart wallets 51
20. Administrators can execute the same fee withdrawal multiple times 53
21. Protocol fees can be claimed only once 55
22. Liquidation rollback can fail 56
23. Protocol pays gas fees for withdrawals to Ethereum 58

24. Lack of validation during pool construction 59
25. borrowing_disabled flag is present but unused 60
A. Vulnerability Categories 61
B. Code Maturity Categories 63
C. Code Quality Recommendations 65
D. Recommendations for Architectural Safety 66
E. Fix Review Results 68
Detailed Fix Review Results 70
F. Fix Review Status Categories 73
About Trail of Bits 74
Notices and Remarks 75

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Bo Henderson, Consultant
bo.henderson@trailofbits.com

Tjaden Hess, Consultant
tjaden.hess@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 19, 2025	Pre-project kickoff call
September 4, 2025	Status update meeting #1
September 9, 2025	Delivery of report draft
September 15, 2025	Report readout meeting
September 19, 2025	Delivery of final comprehensive report
November 14, 2025	Completion of fix review

Executive Summary

Engagement Overview

Liquidium engaged Trail of Bits to review the security of its cross-chain lending protocol, which uses the Internet Computer Protocol (ICP) blockchain's Chain Key technology.

A team of two consultants conducted the review from August 25 to September 8, 2025, for a total of four engineer-weeks of effort. Our testing efforts focused on reviewing access controls for the ICP canister, soundness of cross-chain transfers, maintenance of proper invariants for overcollateralized lending, and availability for prompt liquidations. With full access to source code and documentation, we performed static and dynamic testing of the target codebase, using automated and manual processes.

Observations and Impact

We identified four high-severity issues that could allow funds to be stolen from arbitrary users or from the liquidity pool as a whole (TOB-LIQUIDIUM-1, TOB-LIQUIDIUM-2, TOB-LIQUIDIUM-10, TOB-LIQUIDIUM-13). These issues arise from improper user authentication, calculation of interest, validation of deposit statuses, and handling of pending withdrawals.

We also identified several issues that could cause user funds to be stuck due to accounting issues in error conditions or when the system is under load (TOB-LIQUIDIUM-3, TOB-LIQUIDIUM-4, TOB-LIQUIDIUM-9, TOB-LIQUIDIUM-16).

Generally, the combination of asynchronous and fallible cross-chain transfers with ICP's partial state rollbacks makes any untested error case a potential source of dropped funds or double-spends. The system also lacks a consistent journaling framework, which means that detailed investigation is required for each individual cross-canister call to ensure that partial state rollbacks do not leave the system in a dangerous state.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Liquidium take the following steps before deploying the protocol to production:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Use sub-accounts to handle pending user funds.** Cross-chain transfer tracking is complex and error-prone. By burning and minting directly from and into per-profile sub-accounts, the risk of both theft and accidental loss is mitigated. See appendix D for details.

- **Minimize code that touches core liquidity.** Minimize and clearly delineate components that handle core liquidity, including collateral balances, debt balances, interest accruals, and liquidations. These components require careful accounting to ensure that funds cannot be drained protocol-wide. See appendix D for details.
- **Minimize panicking assertions.** Rust panics cause a Wasm trap condition, which causes a partial rollback of the canister state. The codebase currently uses `assert`, `panic`, `expect`, and `unwrap` statements liberally, even for reachable conditions in functions that return a top-level `Result` type. Remove panic conditions except in considered, documented cases. You can use the Trail of Bits Semgrep lints to identify these conditions:

```
semgrep --config "p/trailofbits"
```

- **Monitor cycle balances and create a plan for cycle drain attacks.** ICP's reverse gas model allows malicious actors to create a denial-of-service condition that may prevent timely liquidations. Automate monitoring for this condition so that funds may be quickly added to top up the cycle balance.
- **Improve testing of error conditions.** Code that handles value transfers or balance updates during an error condition must be validated by testing.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	4
Medium	8
Low	5
Informational	8
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	1
Authentication	1
Data Validation	14
Denial of Service	3
Timing	3
Undefined Behavior	3

Project Goals

The engagement was scoped to provide a security assessment of the Liquidium cross-chain lending protocol. Specifically, we sought to answer the following non-exhaustive list of questions:

- Does cross-chain account management prevent unauthorized access to user funds?
- Does the system preserve invariants under all asynchronous message orderings?
- Can reachable error conditions produce dangerous, invalid states?
- Are funds always accounted for one-to-one between deposits, withdrawals, and user balances?
- Is the system vulnerable to financial attacks or rounding errors?
- Can at-risk positions always be liquidated in a timely manner?
- Are protocol fees correctly computed and safely claimable?
- Can malicious users drain cycles to create a denial of service?
- Are function inputs properly validated before being used?

Project Targets

The engagement involved reviewing and testing the following target.

liquidium-pool

Repository	https://github.com/Liquidium-Inc/liquidium-pool
Version	`5f8185912611629923f02c40345c8e8066c0e38c`
Type	Rust, Wasm
Platform	ICP

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Use of Clippy, Semgrep, and Dylint to identify problematic Rust patterns.
- Comprehensive manual review of the core components including the central lending canister, pool canisters (ERC-20 and BTC), and cross-chain integration logic.
    - We traced execution paths through the codebase to identify invalid state during asynchronous operations. In particular, we checked for inconsistencies between the internal accounting and asynchronous movement of funds across Chain Key and external layer 1 assets.
    - We reviewed the error paths, cancellation flows, and rollback procedures to identify ways in which possible errors could cause inconsistent states or a loss of funds.
    - We compared the interest rate arithmetic against the strategies implemented by similar, well-established protocols such as Aave. We investigated inconsistencies and searched for ways that interest might be misallocated to lenders and borrowers.
    - We reviewed all access controls and the user verification system, looking for ways that the intended authentication strategy could be subverted or ways in which the current implementation was insufficient.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The Solana Program Library (SPL) token pool canister was not reviewed due to its incomplete state and the expectation that it will not be included in the initial deployment.
- We did not review any off-chain components, such as the liquidation bot, user interface, or WalletConnect integration.
- The ICP Chain Key canister implementations and other third-party dependencies were considered out of scope.
- We did not review governance mechanisms or key management practices for the canister administrator keys.

- While we reviewed interest and liquidation calculations for rounding issues, we did not focus deeply on this area as we found the high-level calculations to be incorrect.
- While we attempted to identify all critical areas in which message ordering or panics could cause loss of funds, we did not fully analyze behavior under extreme conditions such as mid-execution traps due to cycle depletion.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The implementation suffers from severe flaws in interest rate calculations (TOB-LIQUIDIUM-2); current spot rates are applied linearly across time periods instead of using a proper global lending index. The arithmetic as a whole would benefit from more systematic specification with mathematical formulas alongside the implementation. Although balances and prices are scaled to 1e27 to manage rounding risks, scaling across the system as a whole is inconsistent, is poorly documented, and frequently uses number literals where named constants would be more appropriate.	**Weak**
Auditing	Logs are produced for important events in the core protocol. We did not review any off-chain monitoring infrastructure, including cycle balance monitoring or outflow imbalance monitoring.	**Further Investigation Required**
Authentication / Access Controls	Critical access control vulnerabilities were identified: the `withdraw` and `add_account` methods lack proper authorization checks (TOB-LIQUIDIUM-1), and bitcoin deposits lack authentication of the intended recipient profile (TOB-LIQUIDIUM-12).	**Weak**
Complexity Management	The codebase demonstrates poor separation of concerns between cross-chain transfer logic and core lending protocol operations. Error handling strategies are inconsistent and ad hoc, with numerous instances in which operations can fail in ways that trap user funds or leave positions in irrecoverable states. The intermingling of asynchronous cross-chain operations with atomic lending logic creates complex pending states with nonobvious failure modes. Dead code exists without documentation explaining its purpose or removal	**Weak**

	timeline (e.g., TOB-LIQUIDIUM-25). Panicking assertions are common in functions that return `Result` types, leading to unhandled rollbacks and unexpected error behavior.	
Decentralization	The current system architecture relies heavily on centralized components and admin privileges. Pool parameters can be modified without timelock mechanisms. However, the governance mechanisms were not in scope for review.	**Further Investigation Required**
Documentation	The README provides no instructions for running tests or setting up development environments. Inline code comments are inconsistent and sometimes outdated, failing to explain complex business logic or architectural decisions. Critical system specifications are missing, particularly around interest rate calculations and cross-chain failure handling procedures. API documentation lacks sufficient detail about error conditions and edge cases.	**Weak**
Testing and Verification	Critical test coverage is missing for authentication flows, error handling paths, and cross-chain failure scenarios. Integration tests exist, but they manually queue required events rather than relying on canister logic, masking problems such as TOB-LIQUIDIUM-21.	**Weak**
Transaction Ordering	Timing-dependent race conditions exist that can trap user funds, particularly around cross-chain transaction processing and timeout handling. The 60-second timeout for repayment processing creates unnecessary user risk during high load periods. Some operations have intra-transaction ordering dependencies that could affect fairness under batch processing scenarios.	**Weak**

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Lack of authorization check in withdraw and add_account methods	Access Controls	High
2	Interest calculations allow pool funds to be drained	Data Validation	High
3	Withdrawal rollback may fail if all funds are withdrawn	Data Validation	Medium
4	User funds could be trapped due to timeout race	Data Validation	Medium
5	Liquidations may be refunded twice	Data Validation	Informational
6	Failed borrow produces WithdrawalFailed notification	Data Validation	Low
7	Stale prices cannot be updated	Data Validation	Medium
8	Lack of a delimiter in asset pair symbols	Data Validation	Informational
9	Liquidator funds become stuck when min_collateral_amount cannot be satisfied	Data Validation	Medium
10	Borrow checks do not account for pending borrows	Timing	High
11	Protocol reports incorrect outflow status on borrow failure	Data Validation	Low

12	BTC deposits can be front-run	Authentication	**Medium**
13	Improper BTC withdrawal status checking may allow double spends and lost funds	Data Validation	**High**
14	Pool event notifications may be sent twice	Data Validation	**Informational**
15	Repayment interest waiver calculations violate invariants	Data Validation	**Medium**
16	High transaction volume can cause deposits to be dropped	Timing	**Medium**
17	Liquidation search time grows quadratically with total number of users	Denial of Service	**Low**
18	Removal of an account iterates over all accounts	Denial of Service	**Low**
19	ERC-20 deposits are not compatible with smart wallets	Data Validation	**Informational**
20	Administrators can execute the same fee withdrawal multiple times	Timing	**Informational**
21	Protocol fees can be claimed only once	Undefined Behavior	**Low**
22	Liquidation rollback can fail	Undefined Behavior	**Medium**
23	Protocol pays gas fees for withdrawals to Ethereum	Denial of Service	**Informational**
24	Lack of validation during pool construction	Data Validation	**Informational**
25	borrowing_disabled flag is present but unused	Undefined Behavior	**Informational**

Detailed Findings

1. Lack of authorization check in withdraw and add_account methods

Severity: **High**	Difficulty: **Low**
Type: Access Controls	Finding ID: TOB-LIQUIDIUM-1
Target: `canisters/lending/src/controllers/{lending_controller.rs, account_controller.rs}`	

Description

The Liquidium canister system identifies unique users by a "profile," which is an opaque `Principal` identifier. Each profile is associated with one or more "accounts," which are chain-native addresses (e.g., EVM EOA address, BTC address, etc.). When withdrawing funds, the protocol canister validates a signature over the request but does not check that the account that produced the signature is associated with the profile from which the funds are being withdrawn. Similarly, the `add_account` method allows an account to be associated with a profile so that the account may sign requests on behalf of the profile. However, the canister does not check that the `AddAccountRequest` itself is signed by an account already associated with the profile.

Figure 1.1 shows the entrypoint for the `withdraw` method. Figure 1.2 shows the entrypoint for the `add_account` method. Note that in both cases, while the request signature is validated, the signing account is never checked to be associated with the request principal.

```
#[update]
pub async fn withdraw(profile: Principal, req: SignedRequest<WithdrawRequest>) ->
Result<OutflowDetails, ProtocolError> {
    req.verify()?;
    Protocol::initialize_withdrawal(profile, req.signature_info.account,
req.data).await
}
```

Figure 1.1: The `withdraw` method entrypoint (`liquidium-pool/canisters/lending/src/controllers/lending_controller.rs#21–25`)

```
#[update]
pub fn add_account(profile: Principal, req: SignedRequest<AddAccountRequest>) ->
Result<(), ProtocolError> {
    req.verify()?;
    AccountManager::add_wallet(profile, req.data)
```

```
}
```

Figure 1.2: The `add_account` method entrypoint (`liquidium-pool/canisters/lending/src/controllers/accounts_controller.rs#32–36`)

The `borrow_assets` method does validate the account mapping via a check inside `initialize_loan`. Note that, as shown in figure 1.3, the function exits with an error if the pair `(profile, borrower)` is not already registered. No equivalent check exists for `withdraw` or `add_account`.

```
async fn initialize_loan(profile: Principal, borrower: Account, loan_request:
BorrowAssetRequest) -> Result<OutflowDetails, ProtocolError> {
    ACCOUNTS.with_borrow(|accounts| accounts.get(&(profile,
borrower)).ok_or(ProtocolError::AccountNotFound))?;
```

Figure 1.3: Example of a check that properly authenticates the user (`liquidium-pool/canisters/lending/src/borrowing/borrowing.rs#41–42`)

Exploit Scenario

Alice deposits 1 ETH into the Liquidium protocol to use as collateral. Mallory creates a new ETH key pair and signs a `WithdrawRequest` message using her own private key. She calls the `withdraw` method on the Liquidium canister and supplies Alice's profile alongside her own signed withdrawal request. The canister fails to check that the signer account is authorized to withdraw funds from Alice's profile, allowing Mallory to steal Alice's funds.

Recommendations

Short term, add a `profile` argument to the `SignedRequest::verify` function so that signature verification is always done with respect to a particular profile, not just an account.

Long term, consider removing the `profile` argument from authenticated methods. Have the system look up the authenticated account in the `WALLETS` mapping in order to derive the profile. Consider removing the public visibility of the `SignedRequest::signature_info` field and instead providing a method that returns the account only after signature verification. These API changes will help prevent developers from relying on unvalidated user-provided account information. Additionally, add tests to demonstrate error cases during authentication so that regressions are detected.

2. Interest calculations allow pool funds to be drained

Severity: **High**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-2
Target: `canisters/lending/src/position/position.rs`	

Description

The Liquidium protocol canister computes current collateral by imputing the current interest rate backward to cover the whole interval since the last position update. This allows lenders to artificially increase their profits by briefly manipulating the interest rate upward and then accruing interest as if the rate were inflated for the full lending period.

The highlighted section of figure 2.1 shows the calculation of the `index_ratio` value and includes a comment indicating that this should track the growth of a global borrow index. However, instead of referencing a global index, the code sets the `index_ratio` value to the current spot lending rate. Note that the `else` branch divides and then multiplies the current `lending_rate` value by the `lending_rate_snapshot`, thus leaving the value unchanged.

The `lending_rate` value is computed by a piecewise-linear function of the current pool supply and does not track interest over time.

Ultimately, this logic applies the current rate linearly across the time since the particular position was last updated. Borrow-side interest is not guaranteed to surpass supply-side interest, allowing lenders to drain funds until the protocol is insolvent.

```
/**
 * Calculates a position's collateral + earning up to a certain timestamp
 */
pub fn calculate_collateral(&self, lending_rate: Nat, timestamp: u64) -> (Nat, Nat)
{
...
    // Calculate how much the global borrow index has grown since the last snapshot
    let index_ratio = if self.lending_rate_snapshot == 0u32 {
        lending_rate.clone()
    } else {

self.lending_rate_snapshot.ray_mul(&lending_rate.ray_div(&self.lending_rate_snapshot
))
    };
...
    // Calculate the per second rate and accumulated interest over time
```

```
    let per_second_rate: Nat = index_ratio / (365 * 24 * 60 * 60) as u32;
...
    // Interest accrued over elapsed time
    let interest_accrued = per_second_rate * elapsed_time;
...
    let newly_generated_interest = (deposited.clone() *
interest_accrued).from_ray();
...
    // Calculate earnings
    let total_collateral = deposited.clone() + newly_generated_interest.clone();
    debug!(" --- Post earnings {}", newly_generated_interest);

    (total_collateral, newly_generated_interest)
}
```

Figure 2.1: Collateral interest calculation (`liquidium-pool/canisters/lending/src/position/position.rs#178–226`)

Exploit Scenario

Mallory borrows 100 USDT from the Liquidium pool using profile X. She immediately re-deposits the USDT into a separate profile Y. After 1 year, Mallory uses profile X to borrow another 500 USDT. Assuming the pre-borrow interest rate is 5%, Mallory now owes 605 USDT in profile X.

Mallory's large borrow causes the pool to go beyond the optimal liquidity point, driving the USDT pool interest rate up to 25%. She then calls `withdraw` using profile Y, which triggers a recomputation of the interest accrued to profile Y. She receives 125 USDT. Next, Mallory repays 605 USDT to account X, plus some dust to cover one second of interest on the balance.

After this sequence of operations, Mallory has earned a net income of just under 20 USDT, taken from the Liquidium protocol funds.

Recommendations

Short term, track a global lending index that updates during every protocol operation. Compute position updates based on snapshots of the global index rather than the point-in-time interest rate. Intuitively, the global index should track the current value of 1 unit of currency deposited at the deployment of the protocol.

Long term, document the equations for computing current rates and accrued interest. Add testing that demonstrates the correct calculation of borrowing and lending rates across several accounts and operations. Consider adding randomized invariant testing, exercising the assertion that, given constant asset prices and no initial open positions, no sequence of operations should cause the total balance of the protocol to drop below its initial value.

3. Withdrawal rollback may fail if all funds are withdrawn

Severity: **Medium**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-3
Target: `canisters/lending/src/withdrawal/withdrawals.rs`	

Description

When a user withdraws funds, the withdrawal should be rolled back if the target chain transfer fails. However, the `cancel_withdrawal` method will trap if the withdrawal is for the full balance of a position, causing user funds to become stuck.

Figure 3.1 excerpts the `cancel_withdrawal` method.

```
let position_id = (withdrawal.profile.unwrap(), withdrawal.pool_id);
Protocol::update_position(&position_id, |position_to_update| {
    // Load position
    let mut position = position_to_update.ok_or(ProtocolError::PositionNotFound)?;
    position.update(pool.lending_rate(), pool.borrow_rate(), None);

    // Update earnings and debt stats
    position.deposited_snapshot += withdrawal.amount.clone();

    Ok((Some(position), None::<()>))
})
.unwrap();
```

Figure 3.1: Position refund within `cancel_withdrawal` (`liquidium-pool/canisters/lending/src/withdrawal/withdrawals.rs#197–208`)

Note that if the user profile has no open position in the pool, the function will panic during the `unwrap` call. This can happen when the user has withdrawn all funds, as shown in figure 3.2:

```
fn update_position<T>(
    id: &(Principal, Principal),
    updater: impl Fn(Option<Position>) -> Result<(Option<Position>, Option<T>),
ProtocolError>,
) -> Result<(Option<Position>, Option<T>), ProtocolError> {
    POSITIONS.with_borrow_mut(|positions| {
        // Apply the update function to the position
        let existing = positions.get(id).map(|p| p.0);

        let (position, metadata) = updater(existing)?;
```

```
        // Save the updated position back
        if let Some(position) = &position {
            let should_remove = position.deposited_snapshot == 0u8 &&
position.borrowed_snapshot == 0u8 && position.pending_outflow == 0u8;
            if should_remove {
                positions.remove(id);
            } else {
                positions.insert(*id, Cbor(position.clone()));
            }
        } else {
            positions.remove(id);
        }

        Ok((position, metadata))
    })
}
```

Figure 3.2: Positions are deleted when balance components are zero. (`liquidium-pool/canisters/lending/src/protocol/protocol.rs#376–400`)

This, in turn, is triggered during the execution of `initialize_withdrawal`, as shown in figure 3.3:

```
let position_id = (profile, req.pool_id);
let (_, amount) = Protocol::update_position(&position_id, |position_to_update| {
    let mut position = position_to_update.ok_or(ProtocolError::PositionNotFound)?;
    // Update position
    position.update(pool.lending_rate(), pool.borrow_rate(), None);

    let amount = if req.amount == 0u8 {
        position.deposited_snapshot.clone()
    } else {
        req.amount.clone()
    };

    if position.deposited_snapshot < amount.clone() {
        return Err(ProtocolError::InsufficientFunds);
    }

    // Save updated position earnings and debt stats for hf calculation
    position.deposited_snapshot -= amount.clone();
    Ok((Some(position), Some(amount)))
})?;
```

Figure 3.3: Position update in `initialize_withdrawal` (`liquidium-pool/canisters/lending/src/withdrawal/withdrawals.rs#54–73`)

Note that the withdrawal deducts from the deposit snapshot but does not increase the value of `pending_outflows`. This can cause all component values to be zero. If that happens, the position will be deleted from the map immediately during `initialize_withdrawal` and any refunds will fail.

Exploit Scenario

Alice withdraws her entire USDT pool balance, targeting her smart wallet contract. The USDT pool initiates a transfer, but the transaction fails due to temporary congestion on the Ethereum chain. The USDT pool calls `cancel_withdrawal` to return Alice's funds, but the method traps because the position no longer exists. The USDT pool does not retry failed transactions, so Alice's funds are stuck and uncredited.

Recommendations

Short term, have `initialize_withdrawal` add pending withdrawal balances to `pending_outflows` rather than immediately deducting them from `deposited_snapshot` during the `initialize_withdrawal` operation. This will keep the position alive until the withdrawal is finalized and will align the withdrawal logic with that of borrowing operations.

Long term, add testing to capture rollback behavior due to cross-chain failures. This can be done by mocking the Chain Key canisters during testing to allow withdrawal failures to be manually triggered. Take care when using assertions in confirmation or cancellation methods, as panics or errors in these functions may leave the system in an unexpected state.

4. User funds could be trapped due to timeout race

Severity: **Medium**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-4
Target: `canisters/lending/src/repayment/repayment.rs`	

Description

When processing repayments, the protocol canister requires that the repayment transaction be at most 60 seconds old. If this check fails, the `on_repayment_created` handler fails, and user funds are not credited. However, the funds have already been transferred, causing the coins to become stuck and uncredited.

Figure 4.1 shows the code that handles repayment accounting. Note that this code is triggered *after* the pool canister has received user funds.

```
fn initialize_repayment(repayment: Repayment) {
    PENDING_REPAYMENTS.with_borrow_mut(|repayments| {
        // Check if repayment already exists
        let exists = repayments.contains_key(&repayment.txid);
        if exists {
            log!("Repayment already exists ({})", repayment.txid);
            return;
        }

        // Check that no more than 60 secs passed since the repay tx was built
        let now = time() / 1e9 as u64;

        debug!(" ** Checking time {now} - {}, {}", repayment.timestamp, now -
repayment.timestamp);

        if now - repayment.timestamp > 60 {
            panic!("Invalid repayment tx");
        }
```

Figure 4.1: Panic due to stale transaction (`liquidium-pool/canisters/lending/src/repayment/repayment.rs#42–58`)

Under testing conditions, it is unlikely that the pool canister will take 60 seconds to send a repayment notification. However, it is very possible when the pool canister is under load. Pool canisters trigger repayment notifications on a 60-second timer and process only 100 notifications per trigger. Thus, if the pool is handling more than 100 transactions per minute, this condition is very likely to be triggered.

Exploit Scenario

The price of BTC increases relative to USDT, causing many users to repay BTC-denominated loans before their health factors become risky. Alice triggers a repayment on her balance and sends BTC to the pool canister. The pool canister registers the payment and adds the repayment to the protocol canister event queue. However, at the time of queueing, there are already 150 unprocessed notifications before Alice's. When the protocol canister receives the notification, it traps due to the timeout condition, and Alice's funds remain stuck in the pool canister but uncredited to her account in the protocol canister.

Recommendations

Short term, track processed transactions for a longer period of time, such as for 24 hours. Increase the timeout period accordingly. Make the timeout period configurable so that admins can increase the retention and timeout period as needed when the protocol is under load.

Long term, add testing to demonstrate that repayment transactions cannot be accounted for twice.

5. Liquidations may be refunded twice

Severity: **Informational**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-5
Target: `canisters/lending/src/liquidation/liquidation.rs`	

Description

Liquidation refunds may be processed twice due to improper synchronization of the refund queue processing.

When a liquidation is canceled because the underlying position is no longer liquidatable, the liquidation collateral is refunded to the liquidator. This refund process happens by queuing a call to `refund_liquidations` that is triggered by a 30-second timer.

As figure 5.1 shows, the refund is cleared from the queue only after the `await` statement. If the external canister call is delayed, it is possible that the refund timer will trigger again in the interim, causing the `refund_liquidations` function to be reentered. In this case, the refund will be sent twice.

```
async fn refund_liquidations() {
    let protocol = get_protocol_instance();
    let refunds = protocol.read_notifications_store(|notifications| {
        let range = notifications.iter().take(100).map(|item| (item.0, item.1
.0)).collect::<Vec<_>>();

        range
    });

    for (id, refund) in refunds {
        let pool = Protocol::get_pool(refund.pool_id).expect("pool not found");
        match pool
            .withdraw_underlying(PoolWithdrawUnderlyingRequest {
                account: Principal::from(refund.account).to_text(),
                amount: refund.amount,
            })
            .await
        {
            Ok(_) => protocol.remove_notification(&id),
            Err(err) => {
                ic_cdk::println!("Liquidation refund failed: {:?}", err);
                continue;
            }
        }
    }
```

```
}
```

Figure 5.1: Refund is removed from the queue only after the cross-canister call. (`liquidium-pool/canisters/lending/src/liquidation/liquidation.rs#209–233`)

We rated this finding's severity as informational because we did not identify a specific vector by which the transactions could be forced to synchronize in an exploitable order. However, the ICP platform does not make any guarantees about inter-canister call ordering that prevent this from happening.

Exploit Scenario

The ICP network delays scheduling a cross-canister call to the pool canister's `withdraw_underlying` method. While the call is pending, the refund timer triggers again and reenters the refund loop, causing the pending refund to be processed twice. The liquidator receives double the correct amount of funds, and the Liquidium protocol may become insolvent.

Recommendations

Short term, have the canister remove the notification from the queue before performing a cross-canister call.

Long term, use a journaling pattern for all asynchronous calls. Before making any cross-canister call, register an intent to do so in a global registry. After the call, either mark the intent as complete or roll back the changes, depending on the call return status.

6. Failed borrow produces WithdrawalFailed notification

Severity: **Low**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-6
Target: `canisters/btc_pool/src/lending/tasks.rs`	

Description

The BTC pool canister triggers a `PoolEvents::WithdrawalFailed` notification rather than a `PoolEvents::BorrowFailed` notification when a transfer corresponding to a borrow operation fails to be included in the BTC blockchain. This causes the protocol canister to add the refunded tokens to the position's collateral rather than subtracting from the pending outflows. In addition, the refunded tokens are credited to the liquidity pool's total supply rather than decreasing the pending debt. This causes the available supply to be artificially inflated, decreasing the interest rates inappropriately.

Note that in the `WithdrawalStatus::Confirmed` branch, the canister checks whether the withdrawal is a borrow or withdrawal (figure 6.1). This check is not present in the `WithdrawalStatus::Failed` branch.

```
match status {
    WithdrawalStatus::Confirmed(txid, _) => {
        // Set the txid
        withdrawal.txid = Some(txid.to_bytes().to_lower_hex_string());

        // Notify borrow confirmation
        match withdrawal.outflow_type {
            OutflowType::Borrow => pool.queue_notification(&id,
&PoolEvents::BorrowConfirmed(withdrawal)),
            OutflowType::Withdraw => pool.queue_notification(&id,
&PoolEvents::WithdrawalConfirmed(withdrawal)),
            _ => {}
        }

        // Remove the item from the processing list
        <BtcPool as Store<_, OutflowDetails>>::remove(&pool, &id);
    }
    WithdrawalStatus::Unconfirmed => {}
    // If the withdrawal failed we need to schedule an account balance rollback
    WithdrawalStatus::Failed(reason) => {
        log!("Withdrawal failed: {reason}");
        // Queue a reimbursement notification
        pool.queue_notification(&id, &PoolEvents::WithdrawalFailed(withdrawal));
        // Remove the item from the processing list
```

```
        <BtcPool as Store<_, OutflowDetails>>::remove(&pool, &id);
    }
```

Figure 6.1: `process_withdrawals` does not distinguish withdrawal type on failure. (`liquidium-pool/canisters/btc_pool/src/lending/tasks.rs#123–146`)

Exploit Scenario

A user borrows BTC, but the Chain Key transfer fails due to an out-of-cycles error in the Chain Key ledger canister. When the refund is triggered, funds are credited to the user and the pool's total supply rather than to the pending outflows. The pool then overestimates the available liquidity and charges too little interest to borrowers.

Recommendations

Short term, add a check to distinguish between withdrawal types during withdrawal failure handling.

Long term, add testing for cancellation conditions across all token pools.

7. Stale prices cannot be updated

Severity: **Medium**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-7
Target: `canisters/lending/src/price_oracle/mod.rs`	

Description

The `sync_price` function incorrectly uses a less-than operator to check whether the cached price is eligible to be synced, preventing stale prices from being updated.

```
async fn sync_price(base: String, quote: String) -> Result<(Nat, u32), String> {
    use crate::price_oracle::exchange_rate_canister::{get_exchange_rate, Asset,
AssetClass, GetExchangeRateRequest};

    let cache = PRICE_CACHE.with_borrow(|cache| cache.get(&format!("{}{}", base,
quote)).clone());

    // 1 minute in ns
    const CACHE_EXPIRY_NS: u64 = 60_000_000_000u64;

    if let Some(entry) = cache {
        if entry.2 < time() - CACHE_EXPIRY_NS {
            return Ok((Nat::from_str(&entry.0).unwrap(), entry.1));
        }
    };
```

Figure 7.1: An incorrect comparison (`liquidium-pool/canisters/lending/src/price_oracle/mod.rs#L51–L63`)

Exploit Scenario

The protocol is deployed, and the initial prices are fetched and saved to the cache. One day later, the timer established by the `register_price_timer` function triggers the next price update. The saved price has a timestamp from 24 hours ago, which is a smaller value than the current time minus the cache expiry constant, which is the timestamp from 1 minute ago. Therefore, the `sync_prices` function will return early instead of syncing prices on this and all future attempts at syncing prices. The protocol must be upgraded to resolve this problem.

Recommendations

Short term, replace the less-than operator highlighted in figure 7.1 with a greater-than operator.

Long term, ensure that all comparisons are verified by unit tests.

8. Lack of a delimiter in asset pair symbols

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-8
Target: `canisters/lending/src/price_oracle/mod.rs`	

Description

The symbol used as a key for saving and loading cached price values lacks a delimiter, introducing the potential for collisions if the protocol is updated to support certain additional assets in the future.

```
let price = rate.rate as u128;
let decimals = rate.metadata.decimals;
let symbol = format!("{}{}", base, quote);

let scale_diff = PRICE_SCALE - decimals as u32;
let scaled_price = Nat::from(price) * 10u128.pow(scale_diff);

PRICE_CACHE.with_borrow_mut(|price_cache| price_cache.insert(symbol,
(scaled_price.to_string(), PRICE_SCALE, time())));
```

Figure 8.1: Use of the error-prone asset pair symbol to save cached prices (`liquidium-pool/canisters/lending/src/price_oracle/mod.rs#L78–L85`)

Exploit Scenario

As part of future updates, support for prices that use the fiat USD currency as a quote asset and for tBTC (wrapped bitcoin on Ethereum) is added. When these asset symbols are concatenated to form a symbol while saving or loading the price of USD in terms of tBTC, the resulting symbol, USDTBTC, collides with the existing symbol used to designate the price of USDT in terms of BTC. If the price of USDT diverges from that of USD, or the price of tBTC diverges from that of BTC, then incorrect prices will be used, resulting in erroneous liquidations or an inability to liquidate unhealthy positions.

Recommendations

Short term, add a delimiter to the symbol between the base and quote assets. This delimiter should be a character that does not commonly occur in asset symbols, and asset symbols should be sanitized to ensure such symbols are not present. To facilitate this, add a `get_price_symbol` helper function to implement the required logic.

9. Liquidator funds become stuck when min_collateral_amount cannot be satisfied

Severity: **Medium**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-9
Target: `canisters/lending/src/liquidation/liquidation.rs`	

Description

Liquidators may specify a minimum amount of collateral to receive during a liquidation in order to provide a profitability floor for the operation. However, if the collateral floor is not achieved, the protocol canister traps. This rolls back the queued refund for the liquidator's deposit, causing the liquidator's funds to be lost without compensation.

Figure 9.1 shows the attempted queue operation for the liquidator refund inside of `Protocol::Liquidate`.

```
// Check if the liquidation amounts are valid
if net_received_collateral <
params.min_collateral_amount.clone().unwrap_or(Nat::from(0u8)) {
    // Queue a rollback
    get_protocol_instance().queue_notification(
        &deposit,
        &LiquidationWithdrawalRollback {
            pool_id: debt_pool.principal,
            amount: amounts.debt_repaid.clone(),
            account: liquidator.to_text().into(),
        },
    );
    return Err("No collateral to liquidate".into());
```

Figure 9.1: Attempted queueing of rollback notification (`liquidium-pool/canisters/lending/src/liquidation/liquidation.rs#295–306`)

After this `Err` value is returned, the top level `liquidate` method panics due to the use of `expect`:

```
#[update]
pub async fn liquidate(account: Principal, params: LiquidationRequest) ->
LiquidationResult {
    Protocol::liquidate(account, params).await.expect("Could not execute
liquidation")
}
```

Figure 9.2: Panic in `liquidate` causes a canister trap. (`liquidium-pool/canisters/lending/src/controllers/liquidation_controller.rs#12–15`)

The ICP Rust CDK treats panics as canister traps, rolling back all state updates to the previous inter-canister call. Thus, the notification queue is rolled back and the liquidation refund is not recorded.

Exploit Scenario

A liquidator calls the `liquidate` method to liquidate a 0.001 BTC debt position against the borrower's USDT collateral. The liquidator specifies a minimum collateral of 1,000 USDT. Due to a price update, the BTC debt is only valued at 995 USDT during liquidation. The liquidation function traps, and the liquidator loses 0.001 BTC without receiving any USDT.

Recommendations

Short term, have the top-level `liquidate` function handle `Err` values by returning a `LiquidationResult` with `LiquidationStatus::Failed`, rather than panicking on errors.

Long term, add tests to ensure the correct operation of liquidation refunds in typical error cases. Review all uses of `expect`, `unwrap`, and `panic` to ensure that no state updates are unexpectedly reverted.

10. Borrow checks do not account for pending borrows

Severity: **High**	Difficulty: **Medium**
Type: Timing	Finding ID: TOB-LIQUIDIUM-10
Target: `canisters/lending/src/borrowing/borrowing.rs`	

Description

When new borrow positions are opened, the protocol canister does not account for pending borrows when checking liquidity limits. Borrows that cannot be confirmed may be initiated; however, this failure occurs after funds have already been sent to the user. Borrowed funds remain accounted for as pending outflows rather than debt. These outflows can never be cleared, which prevents users from fully withdrawing all collateral and prevents the protocol from accruing interest on the borrowed funds. These positions also cannot be liquidated, which could cause the protocol to become insolvent.

Figure 10.1 shows the relevant code in `confirm_loan`. The highlighted condition can fail if multiple loans are initialized concurrently and cumulatively exceed the borrow cap. Similarly, an assertion inside `pool.borrow_funds`, shown in figure 10.2, can trigger if loans that cumulatively overdraw the total liquidity in the pool are issued concurrently. In this case, loans may start to consume protocol fees.

```
fn confirm_loan(id: String) -> Outflow {
    debug!(" ***** Borrowing funds *****");
    // Get the pending loan
    let loan = PENDING_OUTFLOWS.with_borrow(|al| {
        let loan = al.get(&id);
        loan.map(|l| l.0.clone())
    });

    let loan = loan.expect("could not find loan");

    let amount = loan.amount;
    let pool_id = loan.pool_id;
    let receiver_profile = loan.profile.expect("expected profile");

    let mut pool = Protocol::get_pool(pool_id).expect("could not find pool");

    // Check the borrow cap
    if let Some(borrow_cap) = pool.borrow_cap.clone() {
        assert!(pool.total_debt.clone() + amount.clone() <= borrow_cap, "borrow cap
exceeded");
    }
```

```
    // Find an existing position or create a new one
    debug!("Getting position {:?} {:?}", receiver_profile, pool_id);

    let position_id = (receiver_profile, pool.principal);
    Protocol::update_position(&position_id, |position_to_update| {
        // Load position
        let mut position =
position_to_update.ok_or(ProtocolError::PositionNotFound)?;

        // Update earnings and debt stats
        debug!(" ** Updating position {}", amount);

        // Update earnings and debt stats
        position.update(pool.lending_rate(), pool.borrow_rate(), None);

        // Remove the pending outflow
        position.pending_outflow -= amount.clone();

        // Set the borrowed value
        position.borrowed_snapshot += amount.clone();

        Ok((Some(position), None::<()>))
    })
    .unwrap();

    // Update the pool debt
    pool.borrow_funds(amount);
```

Figure 10.1: Borrow cap assertion in `confirm_loan` (`liquidium-pool/canisters/lending/src/borrowing/borrowing.rs#189–235`)

```
assert!(
    amount <= pool.available_liquidity(),
    "Not enough liquidity in the pool {} > {}  ",
    amount,
    pool.available_liquidity()
);
```

Figure 10.2: Panic in `borrow_funds` (`liquidium-pool/canisters/lending/src/pools/pool.rs#120–127`)

Exploit Scenario

Suppose the BTC pool has a borrow cap of 10 BTC, the outstanding pool debt is 5 BTC, and the price of BTC is 100,000 USDT. Mallory, a malicious user, supplies 1,000,000 USDT as collateral. She then initiates a borrow for 1 BTC and then shortly thereafter a second borrow for 5 BTC. Mallory receives the 6 BTC in her BTC wallet.

The BTC pool sees that the transactions have cleared on the Bitcoin network and queues both transactions for `confirm_loan`. The first transaction confirms successfully, and the BTC pool debt is updated to 6 BTC. When handling the second transaction, `confirm_loan`

fails due to the supply cap assertion. At this point, Mallory has 6 BTC in her wallet, 1 BTC accounted for in her `borrowed_snapshot`, and 5 BTC accounted for in `pending_outflow`.

Mallory then repays the 1 BTC debt and deposits the other 5 BTC as collateral. She is then able to repeat the operation several more times, until she has 100 BTC in collateral and 100 BTC in pending outflows. She is then able to indefinitely accrue interest on the 100 BTC in collateral despite not providing any liquidity. She can also borrow large amounts of currency against the synthetic collateral while avoiding liquidation, as pending outflows are not counted as debt for the purpose of liquidation.

Recommendations

Short term, have the system finalize borrowing and withdrawal operations before sending any funds. Canceled outflows may be handled as a repayment with interest waived.

Long term, refactor the protocol canister to separate cross-chain transfers from core liquidity protocol logic. Deposits should be credited in the core protocol only after the cross-chain transfer is complete, and cross-chain withdrawals should be sent only after the core protocol debits are complete. Rollbacks should be treated as separate atomic transactions.

11. Protocol reports incorrect outflow status on borrow failure

Severity: **Low**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-11
Target: `canisters/lending/src/borrowing/borrowing.rs`	

Description

When processing a borrow cancellation, the protocol canister reports the outflow as having the status `OutflowStatus::Success`, as shown in the highlighted portion of figure 11.1.

```
fn cancel_loan(id: String) -> Outflow {
    PENDING_OUTFLOWS.with_borrow_mut(|al| {
        let loan = al.get(&id).expect("Could not find loan").0;

        let position_id = (loan.profile.expect("expected profile"), loan.pool_id);
        Protocol::update_position(&position_id, |position_to_update| {
            // Load position
            let mut position =
position_to_update.ok_or(ProtocolError::PositionNotFound)?;

            position.pending_outflow -= loan.amount.clone();

            Ok((Some(position), None::<()>))
        })
        .unwrap();

        // Revert pending debt
        Protocol::update_pool(&loan.pool_id, |pool_to_update| {
            let mut pool = pool_to_update.expect("could not find pool");
            pool.pending_debt -= loan.amount.clone();

            (Some(pool), None::<()>)
        });

        let outflow = Outflow {
            status: OutflowStatus::Success,
            ..loan
        };
        al.insert(id, Cbor(outflow.clone()));
        outflow
    })
}
```

Figure 11.1: Incorrect outflow status in `cancel_loan` (`liquidium-pool/canisters/lending/src/borrowing/borrowing.rs#248–278`)

A misreported status could cause users and administrators to be confused about the state of the funds.

Exploit Scenario

Alice borrows 1 BTC from the Liquidium protocol. The transaction on the Bitcoin blockchain fails to be included in the chain, and the system issues a refund. However, Alice sees in a UI that the outflow was successful and publicly raises concerns, causing reputational harm to the Liquidium system.

Recommendations

Short term, use `OutflowStatus::Errored` for canceled transactions.

Long term, add tests to ensure correct operation in error conditions.

12. BTC deposits can be front-run

Severity: **Medium**	Difficulty: **High**
Type: Authentication	Finding ID: TOB-LIQUIDIUM-12
Target: `canisters/btc_pool/src/lib.rs`	

Description

Malicious ICP boundary nodes or other network observers may use signed bitcoin transactions from honest users to credit their own Liquidium profile. When a user deposits BTC as collateral or repayment, the user signs a BTC transaction with an output spending to the BTC pool's ckBTC address. The user sends this signed transaction as a parameter to the BTC pool's `deposit` method. However, the signed transaction does not include any information about the intended recipient of the credited funds; therefore, anyone who observes the signed transaction data can front run the ICP update call and submit the transaction as a credit to their own account.

Exploit Scenario

Alice wants to deposit 1 BTC as collateral to her Liquidium portfolio. She signs a transaction outputting 1 BTC to the Liquidium BTC pool deposit address and sends a canister update message to an ICP boundary node. Mallory is a malicious actor running the ICP boundary node. She maliciously drops Alice's update call and sends her own using the same serialized signed transaction. The pool canister broadcasts the transaction and credits Mallory's portfolio while receiving Alice's funds.

Recommendations

Short term, use sub-account-specific BTC deposit addresses derived by providing the sub-account identifier in a call to the ckBTC minter's `get_btc_address` endpoint.

Long term, add user-facing documentation for how users can verify deposit addresses using the `dfx` CLI without relying on the potentially untrustworthy web UI.

13. Improper BTC withdrawal status checking may allow double spends and lost funds

Severity: **High**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-13
Target: `canisters/btc_pool/src/withdrawals/ck_processor.rs`	

Description

The BTC pool canister sends withdrawals using the ckBTC minter's `retrieve_btc_with_approval` method, and then checks whether the transaction was accepted on the BTC chain using the `retrieve_btc_status_v_2` method. However, the handling of the withdrawal status may cause refunds when funds were successfully sent or infinitely delay refunds for transfers that were canceled.

The possible statuses for a BTC withdrawal are shown in figure 13.1. The handling logic in the BTC pool is shown in figure 13.2.

`retrieve_btc_status_v2(block_index: nat64)`

The status of a BTC retrieval request can be checked using this function. The different possible statuses are:

- `Unknown`: There is no information available on the ckBTC minter because there is no retrieval request associated with the given block index, or the retrieval request is old and the corresponding information has already been removed.
- `Pending`: The BTC retrieval request is queued.
- `Signing`: The BTC retrieval request is acquiring all signatures to serve the BTC retrieval request.
- `Sending`: The Bitcoin transaction serving the BTC retrieval request is being sent.
- `Submitted`: The Bitcoin transaction has been transmitted. The transaction ID is returned.
- `AmountTooLow`: The BTC retrieval request could not be served because the Bitcoin miner fees are prohibitively high.
- `Confirmed`: The Bitcoin transaction serving the BTC retrieval request is confirmed inside the ckBTC minter, which happens when the transaction has at least the minimum required number of confirmations specified in the `min_confirmations` parameter above.
- `Reimbursed`: There has been an error when processing the BTC retrieval and the ckBTC amount has been reimbursed.
- `WillReimburse`: There has been an eror when processing the BTC retrieval and the ckBTC amount will be reimbursed.

Figure 13.1: BTC transfer statuses (`https://internetcomputer.org/docs/references/ckbtc-reference`)

```
// If the withdrawal failed we need to schedule an account balance rollback
let status = match withdrawal_status {
    RetrieveBtcStatusV2::Unknown => WithdrawalStatus::Failed("Unknown".into()),
    RetrieveBtcStatusV2::AmountTooLow => WithdrawalStatus::Failed("Amount to
low".into()),
    RetrieveBtcStatusV2::Confirmed { txid } => {
        // Set the txid
        let txid = txid.to_bytes().to_lower_hex_string();
```

```
        WithdrawalStatus::Confirmed(txid, None)
    }
    _ => WithdrawalStatus::Unconfirmed,
};
```

Figure 13.2: ckBTC withdrawal status handling logic (`liquidium-pool/canisters/btc_pool/src/withdrawals/ck_processor.rs#51–62`)

If the status is `Unknown`, the withdrawal is refunded. However, this status is produced when a bitcoin transaction is finalized and then pushed out of the ckBTC minter's status queue by further transactions:

```
match self
    .finalized_requests
    .iter()
    .find(|finalized_request| finalized_request.request.block_index == block_index)
    .map(|final_req| final_req.state.clone())
{
    Some(FinalizedStatus::AmountTooLow) => RetrieveBtcStatus::AmountTooLow,
    Some(FinalizedStatus::Confirmed { txid }) => RetrieveBtcStatus::Confirmed { txid
},
    None => RetrieveBtcStatus::Unknown,
}
```

Figure 13.3: ckBTC minter produces `RetrieveBtcStatus::Unknown` when request is too old. (`ic/rs/bitcoin/ckbtc/minter/src/state.rs#733–742`)

Figure 13.4 shows that this condition occurs specifically when more than `MAX_FINALIZED_REQUESTS` new transactions are added to the finalized request queue. `MAX_FINALIZED_REQUESTS` is set to 100.

```
fn push_finalized_request(&mut self, req: FinalizedBtcRetrieval) {
    assert!(!self.has_pending_request(req.request.block_index));

    if self.finalized_requests.len() >= MAX_FINALIZED_REQUESTS {
        self.finalized_requests.pop_front();
    }
    self.finalized_requests.push_back(req)
}
```

Figure 13.4: `ic/rs/bitcoin/ckbtc/minter/src/state.rs#1098–1105`

If more than 100 BTC withdrawals are processed within a single Bitcoin block, it is possible that a withdrawal could be pushed out of the finalized queue before the BTC pool queries its status. In this case, the user will receive a refund in the Liquidium protocol even though the transaction was successful.

In addition, the withdrawal status handling code treats the `Reimbursed` condition as a pending withdrawal. In this case, the user is entitled to a refund but will never receive the credit as the transaction remains perpetually pending. This condition occurs when the

ckBTC minter attempts to construct a transaction with more than 1,000 inputs, which may be rejected as nonstandard by the Bitcoin network.

Exploit Scenario

A malicious user submits a request for a large BTC withdrawal, followed by 100 small withdrawals in short succession. The withdrawals are processed by the ckBTC canister and distributed to the user. However, when the BTC pool queries for the transaction status of the large withdrawal, it receives a `RetrieveBtcStatus::Unknown` response due to an overflow of the status queue. The BTC pool sends a refund notification to the protocol canister, which re-credits the user's collateral position. The user then repeats the process several times, draining the protocol's funds.

Recommendations

Short term, treat `RetrieveBtcStatus::Unknown` as a successful withdrawal and handle `RetrieveBtcStatus::Reimbursed` as a canceled withdrawal. Note that in some instances, an unsuccessful withdrawal will still result in `RetrieveBtcStatus::Unknown`; in those cases, the user funds would become stuck.

Long term, use profile-specific sub-accounts to manage ckBTC deposits and withdrawals. This will ensure that even if transaction statuses are dropped, no user or protocol funds will be permanently lost.

14. Pool event notifications may be sent twice

Severity: **Informational**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-14
Target: `canisters/{btc_pool,erc_pool}/src/lending/tasks.rs`	

Description

Pool events such as deposit and withdrawal confirmations and cancellations may be sent to the protocol canister twice due to incorrect synchronization behavior during event queue processing.

When a cross-chain transaction is confirmed or canceled, the pool canister notifies the protocol canister to handle the finalization or refund. This process happens by queuing a call to `process_event_queue`, which is triggered by a 60-second timer.

As figure 14.1 shows, the notification is cleared from the queue only after the `await` statement. If the external canister call is delayed, it is possible that the timer will trigger again in the interim, causing the `process_event_queue` function to be reentered. In this case, the notification could be sent twice.

```
pub async fn process_event_queue() {
    let pool = get_pool_instance();
    let protocol_canister = pool.get_protocol_canister();
    let events: Vec<(String, PoolEvents)> =
        pool.read_notifications_store(|deposits|
deposits.iter().take(100).map(|item| (item.0, item.1 .0)).collect());

    for (id, event) in events {
        let result = NotificationsClient::notify_pool_event(protocol_canister,
&event).await;

        match result {
            Ok(_) => pool.remove_notification(&id),
            Err(_) => {
                log!("Could not process event {}", id)
            }
        }
    }
}
```

Figure 14.1: Pool event queue sends before removing notification. (`liquidium-pool/canisters/btc_pool/src/lending/tasks.rs#154–170`)

We rated this finding's severity as informational because we did not identify a specific vector by which the transactions could be forced to synchronize in an exploitable order. However, the ICP platform does not make any guarantees about inter-canister call ordering that prevent this from happening.

Exploit Scenario

The ICP network delays scheduling a cross-canister call to the pool canister's `withdraw_underlying` method. While the call is pending, the refund timer triggers again and `process_event_queue` reenters the refund loop, causing the pending refund to be processed twice. The liquidator receives double the correct amount of funds, and the Liquidium protocol may become insolvent.

Recommendations

Short term, have `process_event_queue` remove the notification from the queue before performing a cross-canister call.

Long term, use a journaling pattern for all asynchronous calls. Before making any cross-canister calls, register an intent to do so in a global registry. After the call, either mark the intent as complete or roll back the changes, depending on the call return status.

15. Repayment interest waiver calculations violate invariants

Severity: **Medium**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-15
Target: `canisters/lending/src/repayment/repayment.rs`	

Description

The Liquidium protocol grants a refund on interest incurred on a borrow position during the time when a repayment transaction is pending. The protocol also allows for repayment transactions with values greater than the total outstanding debt of a position. In this case, the interest refund calculation uses the full value of the repayment transaction rather than the proportion that actually covers outstanding debt. This causes the protocol fee to be slightly underpaid, as the protocol fee is deducted from the interest on the borrowed funds. In extreme circumstances, the debt refund may even be larger than the entire outstanding debt balance of the user.

We also noted that the interest waiver amount is discontinuous; if the repayment transaction clears within the grace period, the interest during the repayment period is waived. However, if the repayment takes longer than the grace period, no waiver is granted at all. A more intuitive formulation would be to simply cap the waived debt.

Finally, the interest waiver system adds risk by violating the invariant that interest accrued by lenders is credited only on debt that is earning interest from borrowers. Because lenders may update their balances during the grace period, they will receive interest on debt during that grace period. When that interest is later waived, it produces a theoretical imbalance that could lead to draining of funds. In practice, the grace periods are likely short (on the order of 1 minute), so the profit accrued in this manner will be quite small and overwhelmed by cross-chain transfer fees. However, if interest rates are very high, the grace period is long, or the values involved are very large, this strategy may be profitable.

Exploit Scenario

Mallory, a malicious user, supplies a large quantity of BTC as collateral to the pool using profile P. She then borrows a large amount of USDT in a different profile Q and immediately repays it. Just before the repayment transaction clears, she triggers a small withdrawal from her lending profile P. This causes the canister to recompute her balance and credit the interest accrued so far. The repayment then goes through, and all interest on the large debt is repaid. Mallory keeps the accrued interest in profile P while avoiding any interest in profile Q. She repeats this until the protocol funds are drained.

Recommendations

Short term, consider disabling repayment interest waivers. The typical repayment values for honest users are small but could be abused to bypass fees or produce outsized rewards for malicious users.

Long term, consider refactoring the codebase to separate handling of pending transaction flows from core lending protocol accounting in order to reduce complexity. See appendix D.

16. High transaction volume can cause deposits to be dropped

Severity: **Medium**	Difficulty: **Medium**
Type: Timing	Finding ID: TOB-LIQUIDIUM-16
Target: `ck_canisters/src/account_utils.rs`	

Description

Under high deposit load, user deposits may not be properly detected, causing user funds to be stuck without crediting the user's balance. This is because the index canister returns the transaction list in most-recent-first order with a limit on the number of transactions searched. In particular, transactions will be dropped if the `CkIndex` canister registers more than 2,000 transactions in 60 seconds. Both the ERC-20 and BTC pool canisters use the `CkIndex::get_account_transactions` method, shown in figure 16.1, in order to verify incoming deposits:

```
async fn get_transactions(&self, owner_principal: &Principal) ->
Vec<crate::ck_index::TransactionWithId> {
    let result = self
        .get_account_transactions(GetAccountTransactionsArgs {
            account: Account {
                owner: *owner_principal,
                subaccount: None,
            },
            max_results: Nat::from(5000u16),
            start: None,
        })
        .await
        .unwrap()
        .0;

    match result {
        GetTransactionsResult::Ok(res) => res.transactions,
        GetTransactionsResult::Err(err) => {
            panic!("Could not fetch canister ck transactions {}", err.message)
        }
    }
}
```

Figure 16.1: Implementation of `get_transactions` (`liquidium-pool/ck_canisters/src/account_utils.rs#85–105`)

While the `max_results` argument is set to 5,000, an internal limit in the `CkIndex` canister restricts the maximum number of return values to 2,000. Because the `start` field is unset, the returned values begin at the most recent transaction.

If more than 2,000 ICRC-1 mints, burns, or transfers occur after a deposit mint but before the `process_deposits` function triggers, the mint will never be detected.

Exploit Scenario

Alice registers a deposit using the `deposit` method of the ERC-20 pool canister. The `ckBTC` minting canister produces a `Mint` block, which is indexed by the `CkIndex` canister. Due to a price swing, many liquidations occur immediately after this minting; each liquidation involves two ICRC-1 transfers. As a result, the `Mint` block is buried, and Alice loses her deposited funds.

Recommendations

Short term, keep a running index of the last seen block index. Have the index canister request `MAX_BLOCKS_PER_RESPONSE` blocks and set the `start` parameter to the previous head plus `MAX_BLOCKS_PER_RESPONSE` when processing transactions.

Long term, use profile-specific sub-accounts to receive deposits. This will reduce the number of transactions that must be processed and will allow for an escape hatch that prevents funds from being stuck.

17. Liquidation search time grows quadratically with total number of users

Severity: **Low**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-LIQUIDIUM-17
Target: `canisters/lending/src/liquidation/liquidation.rs`	

Description

Liquidators search for opportunities using the `get_liquidation_opportunities` method. The implementation of this method causes the overall complexity of the search process to be quadratic in the total number of open positions. The `get_liquidation_opportunities` method is paginated, such that only a fixed number of opportunities is returned per call to the method; however, each call to this method uses compute time proportional to the total number of users in the system. As the number of users grows, liquidators using any fixed limit size will eventually be unable to perform the search due to cycle limits. Figure 17.1 highlights a line nested within the outer loop, which itself internally iterates over the full set of positions. Figure 17.2 shows the inner loop.

```
POSITIONS.with_borrow(|positions| {
    // Iterate through registered accounts and their wallet mappings
    let iterator = positions.keys().skip(offset as usize);
    for position in iterator {
        // break if we reach the limit
        if processed >= limit {
            break;
        }

        let main_account = position.0;

        if visited.contains(&main_account) {
            continue; // Skip already processed accounts
        }

        // Check health factor first - cheaper operation
        let (health_factor, user_stats) = Protocol::health_factor(main_account,
false);

        debug!("Got health factor {} = {:?}", main_account, health_factor);
        if health_factor < 1000u32 {
            // Only fetch positions and stats if health factor is below threshold

            let now = time() / 1e9 as u64;
            let opportunities: Vec<LiquidateablePosition> =
Protocol::get_account_positions(main_account)
                .iter()
```

```
                .map(|p: &Position| {
                    let mut position = p.clone();
                    let pool = Protocol::get_pool(position.pool_id).expect("Could
not find pool");

                    // Calculate the debt amount
                    (position.borrowed_snapshot, _, _) =
position.calculate_debt(pool.borrow_rate(), now);

                    // Calculate collateral amount
                    (position.deposited_snapshot, _) =
position.calculate_collateral(pool.lending_rate(), now);
                    (position, pool.asset_type, main_account,
pool.liquidation_bonus.into()).into()
                })
                .collect();

            // Add positions only for accounts below threshold
            risky_positions.push(LiquidatebleUser {
                account: main_account,
                health_factor,
                positions: opportunities,
                total_debt: user_stats.debt,
            });
        }

        visited.insert(main_account);
        processed += 1;
    }
});
```

Figure 17.1: Outer loop over all positions (`liquidium-pool/canisters/lending/src/liquidation/liquidation.rs#127–177`)

```
fn get_account_positions(portfolio: Principal) -> Vec<Position> {
    let positions = POSITIONS.with_borrow(|positions| {
        positions
            .range(&(portfolio, Principal::anonymous())..)
            .filter(|(a, _)| a.0 == portfolio)
            .map(|position| position.1 .0.clone())
            .collect::<Vec<Position>>()
    });
    positions
}
```

Figure 17.2: Inner loop over all positions (`liquidium-pool/canisters/lending/src/protocol/protocol.rs#302–311`)

Exploit Scenario

Mallory has a large open position shorting BTC. The price of BTC rises quickly; Mallory does not have enough funds to cover her debt and wants to avoid being liquidated, believing the price will later correct downward. She opens many small positions using sockpuppet

accounts, until validators calling `get_liquidation_opportunities` with the default limit start to receive execution errors. Eventually, the community figures out what is happening and adjusts the limits downward, but by this point Mallory's position is deeply insolvent and the protocol as a whole incurs a large loss.

Recommendations

Short term, modify `Protocol::get_account_positions` to iterate over all pools rather than all positions. Consider changing the outer loop to iterate over all users rather than all positions. Together, these actions will make the total time linear in the number of open positions.

Long term, stress test the system under load. Especially consider liquidator performance, cycle limits during canister upgrades, and Chain Key index/minter behavior under high transaction volumes.

18. Removal of an account iterates over all accounts

Severity: **Low**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-LIQUIDIUM-18
Target: `canisters/lending/src/account/account_manager.rs`	

Description

Removal of an account from a user profile requires iterating over all user accounts in the system. This allows an algorithmic complexity attack that could enable cycle drain attacks or simply increase the system cost. While nonreplicated query methods can safely iterate over all users, replicated update methods should avoid doing so.

```
pub fn remove_wallet(profile: Principal, req: RemoveAccountRequest) -> Result<(),
ProtocolError> {
    let account = Account::from(req.wallet_address);

    let wallet = ACCOUNTS
        .with_borrow(|wallets| wallets.get(&(profile, account)).clone())
        .ok_or(ProtocolError::AccountNotFound)?
        .0;

    let accounts = ACCOUNTS.with_borrow(|accounts| accounts.iter().filter(|a| a.0 .0
== profile).count());

    if accounts == 1 {
        return Err(ProtocolError::CannotRemoveSoleAccount);
    }

    ACCOUNTS.with_borrow_mut(|accounts| {
        accounts.remove(&(profile, account));
    });

    WALLETS.with_borrow_mut(|wallets| {
        wallets.remove(&account);
    });

    Protocol::insert_event(profile, ProtocolEventType::AccountRemoved(wallet));

    Ok(())
}
```

Figure 18.1: Iteration over all accounts in the system (`liquidium-pool/canisters/lending/src/account/account_manager.rs#129–155`)

Cycle drain attacks can prevent liquidations and can also cause mid-transaction traps that leave the system in inconsistent states that may be useful to an attacker.

Exploit Scenario

A malicious user wants to prevent liquidation. She adds a very large number of wallets and then starts removing and re-adding a wallet repeatedly. Each removal uses a large number of cycles such that she can drain cycles from the system without being blocked by request-level rate limiting.

Recommendations

Short term, refactor `remove_wallet` so that it performs its tasks in constant time; consider tracking a counter of accounts per profile or requiring that an account cannot sign to remove itself.

Long term, test the system under load and benchmark cycle usage to find methods that consume cycles disproportionate to the size and number of requests. Consider charging a nominal registration fee before allowing users to create new profiles and limiting the number of accounts per profile.

19. ERC-20 deposits are not compatible with smart wallets

Severity: **Informational**	Difficulty: **N/A**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-19
Target: `canisters/erc_pool/src/lending/lending_pool.rs`	

Description

The ERC-20 deposit flow relies on parsing a top-level Ethereum transaction, which prevents deposits from smart wallets and other smart contracts from being recognized.

```
async fn parse_deposit_transaction(tx_hash: &String) -> (String, Nat) {
    let transaction = EthClient::get_transaction(tx_hash).await.expect("Could not
fetch eth transaction");

    assert!(transaction.to.unwrap().to_string() == *CK_MINTER, "invalid minter
canister");

    // Make sure that the deposit method name is correct depositErc20 (0xdb9751af)
    assert!(
        transaction.input[0..4] == hex::decode("db9751af").expect("Could not decode
method hash"),
        "invalid method"
    );

    let params = vec![
        ParamType::Address,
        ParamType::Uint(256),
        ParamType::FixedBytes(32),
        ParamType::FixedBytes(32),
    ];

    let funding_info = ethabi::decode(&params,
&transaction.input[4..]).expect("Could not decode funding input");
    let amount = funding_info[1].clone().into_uint().expect("could not decode
amount");
    let amount = Nat::from_str(&amount.to_string()).expect("could not decode
amount");
    (transaction.from.to_string(), amount)
}
```

Figure 19.1: ERC-20 deposit flow relies on top-level transaction parsing. (`liquidium-pool/canisters/erc_pool/src/lending/lending_pool.rs#184–206`)

While the current UI relies on WalletConnect-based EOAs, users may expect to be able to execute trading strategies from multisignature smart wallets or other on-chain deployments.

Recommendations

Short term, document that only deposits from EOAs will be recognized. Consider placing a proxy in front of the ERC-20 helper contract to block non-EOA deposits.

Long term, consider using event logs rather than transaction hashes to determine deposit parameters.

20. Administrators can execute the same fee withdrawal multiple times

Severity: **Informational**	Difficulty: **Medium**
Type: Timing	Finding ID: TOB-LIQUIDIUM-20
Target: `canisters/lending/src/withdrawal/withdrawals.rs`	

Description

The `initialize_fee_claim` function records the given `PENDING_OUTFLOWS` after executing two asynchronous calls, introducing a race condition that subverts the initial check that intends to ensure a fee claim cannot be initialized while one is currently pending.

```
async fn initialize_fee_claim(pool_id: Principal, account: Account, nonce:
Option<Nat>) -> Result<OutflowDetails, PoolError> {
    let existing_fee_claim = PENDING_OUTFLOWS.with_borrow_mut(|al| {
        al.iter().find(|item| {
            let outflow = &item.1;
            outflow.pool_id == pool_id && outflow.status == OutflowStatus::Pending
&& outflow.outflow_type == OutflowType::FeeClaim
        })
    });

    assert!(existing_fee_claim.is_none(), "fee claim already in progress");

    let id = shared::uuid::generate_uuid_v4().await;

    let pool = POOLS.with_borrow(|pools| pools.get(&pool_id).unwrap().0.clone());

    let req = PoolWithdrawRequest {
        outflow_type: OutflowType::FeeClaim,
        amount: pool.pending_service_fees.clone(),
        nonce,
        account: account.to_string(),
        id: id.to_string(),
        funding_transaction: None,
    };

    let result = PoolCanisterGateway::withdraw(pool_id,
req.clone()).await.expect("could not claim fees");

    PENDING_OUTFLOWS.with_borrow_mut(|al| {
        let timestamp = time() / 1e9 as u64;
        al.insert(
            id.to_string(),
            Cbor(Outflow {
```

```
                id: id.to_string(),
                amount: req.amount,
                pool_id,
                profile: None,
                outflow_type: OutflowType::FeeClaim,
                receiver: account,
                status: OutflowStatus::Pending,
                timestamp,
            }),
        )
    });
```

Figure 20.1: The `initialize_fee_claim` method (`liquidium-pool/canisters/lending/src/withdrawal/withdrawals.rs#L240–L283`)

The execution flow of the `initialize_fee_claim` method pauses at the two asynchronous calls highlighted in figure 20.1, allowing additional transactions to begin executing and pass the initial check.

Exploit Scenario

Alice, a Liquidium administrator, attempts to collect protocol fees for the first time via a helper script. Due to a bug in this script, the same call to the `claim_fee` method is sent multiple times. All calls succeed, causing Alice to have inadvertently stolen funds from the protocol.

Recommendations

Short term, have `initialize_fee_claim` record that the new pending outflow is being processed immediately after checking that one does not already exist. This will require the UUID to be created at the very beginning of the function, to avoid reentrancy during the asynchronous call.

Long term, thoroughly review the fee claiming mechanism and refactor it according to the patterns described in appendix D.

21. Protocol fees can be claimed only once

Severity: **Low**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-LIQUIDIUM-21
Target: `canisters/lending/src/withdrawal/withdrawals.rs`	

Description

The `FeeClaimConfirmed` and `FeeClaimFailed` variants of the `PoolEvents` enum are never queued. As a result, the status of the stored `PoolWithdrawRequest` is never changed from `Pending` to `Success` or `Error`, blocking any future attempts to claim fees.

Exploit Scenario

Alice, a Liquidium administrator, collects fees for the first time, saving a pending `PoolWithdrawRequest` and withdrawing the funds to the designated wallet. When she attempts to collect fees for the second time, she gets an error message that reads "fee claim already in progress." Further fees cannot be collected until the protocol is upgraded to resolve this problem.

Recommendations

Short term, queue `FeeClaimConfirmed` and `FeeClaimFailed` variants of the `PoolEvents` enum appropriately.

Long term, thoroughly review the fee claiming mechanism. Note that once a cross-chain withdrawal succeeds, it cannot be rolled back. Therefore, any internal accounting that occurs after this point should not throw unrecoverable errors. In addition, the integration tests manually queue the required `PoolEvents`, masking this problem. Review the integration tests and remove such manual interventions. Instead, rely on the canister logic to perform all required actions.

22. Liquidation rollback can fail

Severity: **Medium**	Difficulty: **Medium**
Type: Undefined Behavior	Finding ID: TOB-LIQUIDIUM-22
Target: `canisters/lending/src/liquidation/liquidation.rs`	

Description

If the `revert_borrower_positions` function traps after a withdrawal failure, then the rollback event is not queued, and the liquidator loses the funds used to pay back the targets' debt.

```
// Update borrower positions
Self::update_borrower_positions(&amounts, &debt_position, &collateral_position)?;

// Send the collateral to the liquidator
let mut res = match collateral_pool
    .withdraw_underlying(PoolWithdrawUnderlyingRequest {
        account: liquidator.to_text(),
        amount: net_received_collateral.clone(),
    })
    .await
{
    Ok(res) => Ok(LiquidationResult {
        amounts,
        tx_id: res,
        collateral_asset: collateral_pool.asset_type.clone(),
        debt_asset: debt_pool.asset_type.clone(),
        status: LiquidationStatus::Success,
    }),
    Err(err) => {
        debug!("Withdrawal failed, rolling back positions");

        // Roll back positions
        Self::revert_borrower_positions(&amounts, &debt_position,
&collateral_position)?;

        // Queue a rollback
        get_protocol_instance().queue_notification(
```

Figure 22.1: Error handling logic of the `liquidate` method can trap. (`liquidium-pool/canisters/lending/src/liquidation/liquidation.rs#L309–L334`)

The `revert_borrower_positions` function will trap if either of the given positions no longer exists. This may happen due to generic race conditions; or, if the health factor is less than 0.95 and the liquidator wipes out the borrower's entire position, the `update_borrower_positions` function will remove the given debt position.

```
if debt.deposited_snapshot == 0u8 && debt.borrowed_snapshot == 0u8 &&
debt.pending_outflow == 0u8 {
    positions.remove(&debt.id());
} else {
    positions.insert(debt.id(), Cbor(debt));
}
```

Figure 22.2: The `update_borrower_positions` function will remove the debt position if 100% of debt is repaid by the liquidator. (`liquidium-pool/canisters/lending/src/liquidation/liquidation.rs#L309–L334`)

Exploit Scenario

Alice, a Liquidium lender, takes out a high-risk loan that is on the edge of the liquidation threshold. Following a sudden drop in the price of her collateral, her health factor drops below 0.95, and her loan qualifies for complete liquidation. Bob, a liquidator, deposits tokens sufficient to wipe out Alice's entire debt, and, as a result, the `update_borrower_positions` function removes Alice's zero-debt position from storage. The withdrawal step fails, and the `revert_borrower_positions` function traps and halts execution. Bob receives neither the collateral he expects nor a refund for paying off Alice's debt and loses funds as a result.

Recommendations

Short term, update `revert_borrower_positions` so that it does not return a `Result`. Instead, if one of the positions no longer exists, it should be recreated from the given `debt_position` or `collateral_position` parameters.

Long term, journal intents and test failure cases more thoroughly.

23. Protocol pays gas fees for withdrawals to Ethereum

Severity: **Informational**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-LIQUIDIUM-23
Target: `canisters/erc_pool/src/lending/lending_pool.rs`	

Description

Validation of the funding transaction is commented out in the `handle_withdrawal` function. As a result, users are not required to pay the L1 gas fees while executing withdrawals. Instead, the protocol pays these fees on the user's behalf.

```
async fn handle_withdrawal(req: PoolWithdrawRequest) -> Result<OutflowDetails,
String> {
    // Validate the funding transaction TODO: Tackle funding part
    // Self::validate_funding_transaction(&req.funding_transaction.unwrap()).await;
```

Figure 23.1: The `handle_withdrawal` function does not require a funding transaction. (`liquidium-pool/canisters/erc_pool/src/lending/lending_pool.rs#L112–L114`)

Apart from being a potentially costly expense for the protocol, this opens the system up to denial-of-service attacks that attackers may be motivated to conduct due to complex and unforeseeable interactions between multiple applications. In addition, if support is added for ERC-777 tokens or some other avenue for executing custom code during withdrawals, attackers could use this to steal funds from the protocol by minting gas tokens.

Exploit Scenario

Bob is a user of Liquidium and of other DeFi lending applications on Ethereum. One of these other applications requires Bob to make a time-sensitive movement of USDT to prevent himself from being liquidated. Alice notices that most of Bob's USDT is deposited in Liquidium, and she expects Bob to withdraw his funds soon to keep his other position healthy. Alice makes withdrawals from Liquidium in a loop until the Liquidium gas reserves are depleted or a circuit breaker trips, halting withdrawals. As a result, Bob is unable to access his funds in time, his position becomes unhealthy, and Alice is able to perform a profitable liquidation.

Recommendations

Short term, require users to pay for their own gas during withdrawals by validating funding transactions and adding appropriate tests.

Long term, review and document all outflow of funds from the protocol. Any funding outflows that come at the expense of the protocol should be mitigated.

24. Lack of validation during pool construction

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-LIQUIDIUM-24
Target: `canisters/lending/src/controllers/admin_controller.rs`	

Description

Several pool parameters are scaled values that represent percentages. Some, such as the `protocol_liquidation_fee`, will cause undefined behavior or panics if set to values greater than 100%. Others, such as the `liquidation_threshold`, will allow attackers to drain funds from the protocol if they are over 100%.

Exploit Scenario

Alice, a new Liquidium administrator, creates a new pool with a liquidation threshold of 12,000. As a result, users can borrow more value than the collateral can support.

Recommendations

Short term, add assertions to the `register_pool` method of the `admin_controller` that verifies applicable percentage values are less than or equal to 100%.

Long term, ensure that all parameters provided to all entrypoint methods are properly validated.

25. borrowing_disabled flag is present but unused

Severity: **Informational**	Difficulty: **High**
Type: Undefined Behavior	Finding ID: TOB-LIQUIDIUM-25
Target: `canisters/lending/src/config.rs`	

Description

A getter and setter are defined for the `borrowing_disabled` state variable, but they are never used outside of tests that verify they work as expected.

Exploit Scenario

Alice, a new Liquidium administrator, is alerted to a problem involving invalid borrows. She sees that there is an `update_borrowing_disabled` method on the production canister's interface, so she calls it to set the `borrowing_disabled` flag to true. However, this action has no effect, and her response to this incident is delayed.

Recommendations

Short term, remove this functionality if it is not needed. Otherwise, use it appropriately.

Long term, review the codebase and remove all instances of dead code. This will improve maintainability and decrease the risk of miscommunications.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Code Quality Recommendations

We identified code quality issues through manual and automatic code review. These issues are not associated with any specific vulnerabilities. However, addressing them may improve performance, enhance code readability, and prevent the introduction of vulnerabilities in the future.

- **Prefer named constants over number literals.** For example, the `1000u64` literal at `liquidation.rs#L411` should be replaced with the `FULL_LIQUIDATION_RATIO` constant. In particular, having all scale values defined in one place and used consistently will greatly improve the readability of arithmetic operations. Consider using a fixed-point type to track decimals rather than using ad hoc scaling.
- **Revise incorrect comment.** The specification comment associated with the `get_liquidation_opportunities` function at `liquidation.rs#L75–L79` describes the limit parameter as "Max number of users to return." However, this parameter is actually the maximum offset at which the iteration will terminate. Either update this comment to accurately describe this parameter's role, or initialize the `processed` variable to zero at `liquidation.rs#L123` to bring the implementation in line with the comment's description.
- **Create single-use interfaces.** The `lending.did` interface contains both production methods and dev-hook methods that are used only for testing. Remove these dev-hook methods and create a dedicated `lending-dev.did` interface to clarify that these methods ought not to be included in the production canister.
- **Optimize trivial price operations.** Short-circuit price operations to return the identity price (`1e27`) when the base and quote currencies are identical.

D. Recommendations for Architectural Safety

Intermingling of cross-chain transfer logic with core lending logic is a significant source of complexity and severe issues in Liquidium (TOB-LIQUIDIUM-10, TOB-LIQUIDIUM-11, TOB-LIQUIDIUM-15). The asynchronous and fallible nature of cross-chain transfers currently demands that rollback handling be integrated into core accounting functions and produces a variety of ad hoc pending states with nonobvious properties. Unexpected errors or panics can lead to stuck user funds or unliquidatable positions.

In addition, cross-chain transfer accounting is complex and has resulted in a number of issues (TOB-LIQUIDIUM-13, TOB-LIQUIDIUM-16, TOB-LIQUIDIUM-19) that could cause dropped funds or enable double-spending.

Proposed Architecture

We propose a modified architecture that adheres to the following requirements to ensure that funds are always accounted for:

1. Operations affecting protocol-wide liquidity make no cross-canister calls except for ICRC-2 transfers of Chain Key tokens. Each operation consists of an optional ICRC-2 transfer-from deposit, then profile balance updates, then an optional ICRC-2 transfer withdrawal.

2. Each operation is journaled; at the beginning of each operation, an entry is created in the canister state with a unique identifier. After each of the cross-canister calls, the journal entry is updated to reflect the next operation. In case of an error, a recovery function may later be called, which checks the state of the journal entry and recovers appropriately.

3. Pending deposits and withdrawals, and any other funds that are not actively supplied as collateral, are stored in user-specific sub-accounts within the pool canisters. Chain Key mints and burns target specific user sub-accounts, except in the case of boosted withdrawals, which may be temporarily held in a pooled account until a burn is triggered. Users may manually retrigger withdrawals from their Chain Key balances at any time. This minimizes the risk of stuck funds due to interrupted high-level flows.

To implement these requirements, we suggest splitting the current protocol canister into two distinct canisters:

1. The lending canister will be a minimal, purely ICRC-2 native, overcollateralized lending protocol operating on Chain Key tokens. It will handle all operations that impact the overall liquidity of the lending protocol, including collateral/debt accounting, liquidations, and protocol fees. This canister will be closely reviewed for journaling and rollback best practices. In addition, liquidators will interact directly

with this canister and all methods will be reviewed for cycle efficiency to prevent cycle drain attacks that prevent liquidations.

2. The UX canister will manage cross-chain deposit and withdrawal flows and other end-to-end operations for a single user. The UX canister will be responsible for account management and authentication and for providing early sanity checks, event notifications, and status tracking. The UX canister will handle errors as well as possible, but will be less sensitive to traps as it will never directly handle funds or core protocol state. Cycle accounting is less important, since liquidations will be processed directly through the lending canister.

Journaled Operation Flow Example

We give an example of a journal flow that allows recovery from trap conditions at every step:

1. The UX canister calls `deposit` on the lending canister, supplying a profile, amount, and pool.
2. The lending canister creates a UUID and a `Deposit` structure containing the profile, amount, and pool, and adds it to the journal map using the UUID as a key. The status is initialized to `Incomplete`.
3. The lending canister calls `icrc2_transfer_from` to move funds from the pool sub-account to the global lending pool.
4. If the transfer fails, the `Deposit` journal entry is marked as `Complete` and the transfer returns.
5. Otherwise, the user collateral balance is updated. If this succeeds, the journal entry is marked as `Completed` and the transfer returns. Otherwise, the journal entry is left as `Incomplete` to indicate that the transfer has completed but the balance still needs to be updated.
6. At any time, the UX canister can execute a recovery method to refund an `Incomplete` journal entry. The lending canister then issues an ICRC-2 transfer of the same amount to the original depositor's sub-account in the pool canister and marks the entry as `Complete`.

Completed entries can be removed from the journal immediately or retained for some time period for auditing purposes.

E. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From November 12 to November 14, 2025, Trail of Bits reviewed the fixes and mitigations implemented by the Liquidium team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

While remediating the issues in this report, the Liquidium team undertook a significant rearchitecture and refactoring effort, as recommended in appendix D. This rearchitecture appears to place the system in a substantially stronger security posture. While this fix review verified that the original issues presented in this report have been resolved, we did not review the new implementations end to end. We strongly recommend conducting a follow-up review scoped to cover the full system.

In summary, of the 25 issues described in this report, Liquidium has resolved 24 issues and has not resolved the remaining issue, which was rated as informational severity. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Lack of authorization check in withdraw and add_account methods	High	Resolved
2	Interest calculations allow pool funds to be drained	High	Resolved
3	Withdrawal rollback may fail if all funds are withdrawn	Medium	Resolved
4	User funds could be trapped due to timeout race	Medium	Resolved
5	Liquidations may be refunded twice	Informational	Resolved
6	Failed borrow produces WithdrawalFailed notification	Low	Resolved
7	Stale prices cannot be updated	Medium	Resolved

8	Lack of a delimiter in asset pair symbols	Informational	Resolved
9	Liquidator funds become stuck when min_collateral_amount cannot be satisfied	Medium	Resolved
10	Borrow checks do not account for pending borrows	High	Resolved
11	Protocol reports incorrect outflow status on borrow failure	Low	Resolved
12	BTC deposits can be front-run	Medium	Resolved
13	Improper BTC withdrawal status checking may allow double spends and lost funds	High	Resolved
14	Pool event notifications may be sent twice	Informational	Resolved
15	Repayment interest waiver calculations violate invariants	Medium	Resolved
16	High transaction volume can cause deposits to be dropped	Medium	Resolved
17	Liquidation search time grows quadratically with total number of users	Low	Resolved
18	Removal of an account iterates over all accounts	Low	Resolved
19	ERC-20 deposits are not compatible with smart wallets	Informational	Resolved
20	Administrators can execute the same fee withdrawal multiple times	Informational	Resolved
21	Protocol fees can be claimed only once	Low	Resolved
22	Liquidation rollback can fail	Medium	Resolved

23	Protocol pays gas fees for withdrawals to Ethereum	Informational	Resolved
24	Lack of validation during pool construction	Informational	Resolved
25	borrowing_disabled flag is present but unused	Informational	Unresolved

Detailed Fix Review Results

TOB-LIQUIDIUM-1: Lack of authorization check in withdraw and add_account methods
Resolved in commit `56ff64d`. The `withdraw` and `add_account` methods now validate that the wallet that signs the request is authorized to access the supplied profile.

TOB-LIQUIDIUM-2: Interest calculations allow pool funds to be drained
Resolved in PRs #19 and #24. Users now hold synthetic shares that are purchased and redeemed at values according to a global interest index. This helps ensure that interest growth is applied consistently across time and users.

TOB-LIQUIDIUM-3: Withdrawal rollback may fail if all funds are withdrawn
Resolved in commit `ddc9c96`. Withdrawals no longer support automatic rollback due to cross-chain transfer failures. However, we noted related patterns in the deposit and borrow flows where state rollback may fail when a position is closed out.

TOB-LIQUIDIUM-4: User funds could be trapped due to timeout race
Resolved in PR #27. Repayments no longer time out, preventing dropped repayment transactions.

TOB-LIQUIDIUM-5: Liquidations may be refunded twice
Resolved in PR #11. Event processing is now guarded by a mutex lock, preventing asynchronous reentry of the event queue during processing.

TOB-LIQUIDIUM-6: Failed borrow produces WithdrawalFailed notification
Resolved in commit `ddc9c96`. Failed cross-chain transfers no longer post back to the protocol canister, but rather allow for transfer retries.

TOB-LIQUIDIUM-7: Stale prices cannot be updated
Resolved in PR #15. The comparison operator has been fixed to allow updates to old price cache entries.

TOB-LIQUIDIUM-8: Lack of a delimiter in asset pair symbols
Resolved in PR #15. Quote pair symbols now include an underscore as a delimiter.

TOB-LIQUIDIUM-9: Liquidator funds become stuck when min_collateral_amount cannot be satisfied
Resolved in PR #27. Liquidation requests no longer specify a minimum collateral amount. Liquidation processing occurs using a write-ahead log to prevent stuck funds.

TOB-LIQUIDIUM-10: Borrow checks do not account for pending borrows
Resolved in PR #27. The protocol canister no longer handles pending borrows; pending funds for cross-chain transfers are handled exclusively in the pool canisters.

TOB-LIQUIDIUM-11: Protocol reports incorrect outflow status on borrow failure
Resolved in PR #27. The protocol canister no longer supports pending borrows or loan cancellation.

TOB-LIQUIDIUM-12: BTC deposits can be front-run
Resolved in PR #25. Deposits for all chain types are now processed via subaccounts that encode the destination profile and inflow type. Inflows are detected automatically and credited to the appropriate user based on the BTC deposit address.

TOB-LIQUIDIUM-13: Improper BTC withdrawal status checking may allow double spends and lost funds
Resolved in PR #25. BTC outflows are now processed out of dedicated outflow subaccounts. Outflows are limited to those funds specifically allocated for a particular user, preventing double spends.

TOB-LIQUIDIUM-14: Pool event notifications may be sent twice
Resolved in PR #11. Event processing is now guarded by a mutex lock, preventing asynchronous reentry of the event queue during processing.

TOB-LIQUIDIUM-15: Repayment interest waiver calculations violate invariants
Resolved in commit `be8f2b3`. Repayment no longer handles interest waivers.

TOB-LIQUIDIUM-16: High transaction volume can cause deposits to be dropped
Resolved in PR #25. Inflows are now tracked arbitrarily deep and use both the main and archive ledgers.

TOB-LIQUIDIUM-17: Liquidation search time grows quadratically with total number of users
Resolved in PR #20. The liquidation search now iterates over users rather than open positions, allowing for linear search complexity.

TOB-LIQUIDIUM-18: Removal of an account iterates over all accounts
Resolved in PR #22. Account iteration now only handles accounts associated with a single profile.

TOB-LIQUIDIUM-19: ERC-20 deposits are not compatible with smart wallets
Resolved in PR #25. ERC-20 deposits are now handled through inflow addresses, allowing any user to send funds to the `CkMinter` from any EVM account.

TOB-LIQUIDIUM-20: Administrators can execute the same fee withdrawal multiple times
Resolved in PR #27. Fee claims are processed atomically and then added to a write-ahead log for execution.

TOB-LIQUIDIUM-21: Protocol fees can be claimed only once
Resolved in PR #27. Fee claims are processed atomically and then added to a write-ahead log for execution. The confirmation handling logic has been removed.

TOB-LIQUIDIUM-22: Liquidation rollback can fail
Resolved in PR #27. Liquidation accounting is now handled atomically and failures are handled gracefully to ensure refunds are sent when appropriate.

TOB-LIQUIDIUM-23: Protocol pays gas fees for withdrawals to Ethereum
Resolved in PR #25. A fee is now deducted from ERC-20 withdrawals to cover the ETH native gas fee. A dedicated subaccount fronts the gas fee and accepts payment in ERC-20 tokens. An automated process then swaps the tokens to refill the fee subaccount.

TOB-LIQUIDIUM-24: Lack of validation during pool construction
Resolved in PR #21. Pools now enforce basic range constraints on parameters during construction.

TOB-LIQUIDIUM-25: borrowing_disabled flag is present but unused
Unresolved. The `borrowing_disabled` flag is still present and nonfunctional.

F. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Liquidium under the terms of the project statement of work and has been made public at Liquidium's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit its use.